Data Auditing & Validation Project (Excel + SQL)

Portfolio Case Study by Ryan Swanson

Project Overview

This project expanded an Excel-based provider data audit into SQL using MotherDuck, enabling deeper validation, classification, and KPI reporting. The goal was to identify missing provider identifiers, incomplete contract data, and overall data quality issues using structured SQL queries.

Objectives

- Import and review provider audit data in SQL
- Create a cleaned SQL table with readable field names
- Identify missing NPI values and contract dates
- Group provider records by state and data issue type
- Classify records based on data completeness and risk level
- Calculate an overall data completeness percentage
- Produce query-based summaries to support data quality reporting

Tools Used

- MotherDuck SQL / DuckDB
- SQL queries including SELECT, WHERE, GROUP BY, ORDER BY, COUNT, CASE, IS NULL, IS NOT NULL, and ROUND

Methodology

1. Imported the completed provider audit CSV into MotherDuck
2. Reviewed the imported dataset to inspect structure, fields, and formatting
3. Created a cleaned table with readable column names for easier querying and analysis
4. Queried records with missing NPI values and missing contract dates
5. Used aggregation to count providers by state and summarize data issue categories
6. Applied CASE statements to classify records as Complete, Missing NPI, Missing Contract Date, or higher-risk records
7. Calculated overall data completeness percentage using SQL logic
8. Interpreted results and translated query outputs into audit findings and recommendations

Data Auditing & Validation Project (Excel + SQL)

Portfolio Case Study by Ryan Swanson

Key SQL Analysis Performed

- Previewed imported data using SELECT and LIMIT
- Created a cleaned working table from the imported dataset
- Identified missing provider identifiers using IS NULL
- Counted missing contract date records
- Grouped provider counts by state using GROUP BY
- Classified data quality status using CASE logic
- Summarized missing data by state
- Calculated total records, complete records, and data completeness percentage

Key Findings

- Overall data completeness was calculated at **66.67%**
- Data completeness of 66.67% indicates a significant gap in required field population, which could impact reporting accuracy and downstream analytics
- Approximately one-third of records contained missing critical fields such as NPI or contract date
- Missing provider identifiers and incomplete contract data were the primary data quality concerns
- SQL classification logic helped separate complete records from medium-risk and high-risk records
- Aggregating records by state and issue type made it easier to identify where data quality problems were concentrated

Recommendations

- Implement required-field validation for NPI and contract date fields at the point of data entry
- Use standardized data quality checks before reporting or downstream analysis
- Monitor data completeness percentage as a recurring KPI
- Create recurring SQL validation queries to identify missing or incomplete provider records before they affect reporting accuracy
- Review high-risk records first, especially records missing both provider identifiers and contract information

Data Auditing & Validation Project (Excel + SQL)

Portfolio Case Study by Ryan Swanson

Skills Demonstrated

SQL | Data Analysis | Data Validation | Data Cleaning | Data Quality Analysis | KPI Reporting | Aggregation | CASE Statements | Healthcare Data Insight | MotherDuck / DuckDB | Analytical Thinking

Conclusion

This SQL extension demonstrates the ability to move from Excel-based data auditing into database-driven analysis. By importing the provider dataset into MotherDuck, cleaning the table structure, writing validation queries, classifying record quality, and calculating completeness metrics, this project reflects practical skills applicable to data analysts, healthcare operations, reporting analysts, and quality assurance roles.

Screenshots

Clean Table Creation & Validation

```sql
--Preview imported dataset before cleaning
SELECT *
FROM "provider.data.audit.project"
LIMIT 10;

--Created a cleaned SQL table by restructuring raw imported data and standardizing column names for analysis
CREATE TABLE provider_data_clean AS
SELECT
    column0 AS provider_name,
    "Provider Data Audit Summary" AS npi,
    column2 AS state,
    column3 AS specialty,
    column4 AS status,
    column5 AS contract_date,
    column6 AS data_issue
FROM "provider.data.audit.project"
WHERE column0 != 'Provider Name';
```

```sql
--Validate cleaned table structure
SELECT *
FROM provider_data_clean;
```

9 rows returned in 299ms

	provider_name	npi	state	specialty	status	contract_date	data_issue
1	John Smith	1234567890	NY	Cardiology	Active	1/15/2022	OK
2	Jane Doe	1234567891	NY	Primary Care	Active	11/20/2021	OK
3	Mike Lee	1234567892	CA	Orthopedics	Inactive	NULL	Missing Contract Date
4	Sarah Kim	1234567893	CA	Primary Care	Active	3/10/2023	OK
5	Chris Paul	1234567894	TX	Cardiology	Active	6/25/2020	OK
6	Anna White	NULL	TX	Primary Care	Active	8/30/2022	Missing NPI
7	David Green	1234567896	TX	Orthopedics	Inactive	2/14/2021	OK
8	Lisa Ray	1234567897	NY	Primary Care	Active	NULL	Missing Contract Date
9	Tom Black	1234567898	CA	Cardiology	Active	1/1/2023	OK

Data Auditing & Validation Project (Excel + SQL)

Portfolio Case Study by Ryan Swanson

Data Quality Classification (CASE Statement)

```sql
--Summarize missing critical fields by state
SELECT
    state,
    COUNT(*) AS total_providers,
    COUNT(CASE WHEN npi IS NULL THEN 1 END) AS missing_npi_count,
    COUNT(CASE WHEN contract_date IS NULL THEN 1 END) AS missing_contract_date_count
FROM provider_data_clean
GROUP BY state
ORDER BY missing_contract_date_count DESC, missing_npi_count DESC;
```

3 rows returned in 301ms

	state	total_providers	missing_npi_count	missing_contract_date_count
1	NY	3	0	1
2	CA	3	0	1
3	TX	3	1	0

Data Completeness KPI Calculation (66.67%)

```sql
--Calculate overall data completeness percentage
SELECT
    COUNT(*) AS total_records,
    COUNT(CASE
        WHEN npi IS NOT NULL AND contract_date IS NOT NULL
        THEN 1
    END) AS complete_records,
    ROUND(
        COUNT(CASE
            WHEN npi IS NOT NULL AND contract_date IS NOT NULL
            THEN 1
        END) * 100.0 / COUNT(*),
        2
    ) AS data_completeness_percentage
FROM provider_data_clean;
```

1 row returned in 290ms

	total_records	complete_records	data_completeness_percentage
1	9	6	66.67